Programming Assignment: Unit 8

Hset Paing Htoo

Introduction to Computer Science Program, University of the People

CS 1102-01 Programming 1 AY2026-T5

Instructor : Sirajo Musa (Instructor)

Aug 13, 2026

Employee Data Processing Using the Function Interface and Streams

1. Introduction

For this assignment I built a Java program that processes a dataset of employee records using the Function interface and the Stream API. The program loads a collection of employees, uses a Function to turn each employee into a concatenated name-and-department string, builds a new collection from those transformed values through a stream, calculates the average salary using the stream's built-in reduction operations, and filters the dataset by an age threshold. I developed and tested the program against JDK 21, and it runs from the command line with no external dependencies.

2. Program Design

The program is organized into two classes so that the data and the processing logic stay separate. The Employee class is a plain data holder with the four required attributes -- name, age, department, and salary -- along with accessor methods and an overridden toString() so records print readably. The EmployeeStreamProcessor class contains all the program logic: it loads the dataset, declares the Function and Predicate that the pipelines rely on, and runs each required operation as its own clearly labeled step.

The Function that concatenates the name and department is declared once as a named constant rather than being written inline every time it is needed. This was a deliberate choice: because a Function is a value in Java, defining it once lets the same transformation be reused across several

different stream pipelines and composed with other functions later in the program. In this program the same Function instance is used in four separate places.

Each operation is also placed in its own method with a descriptive name, so the main() method reads as a list of the steps the assignment asked for rather than as one long block of stream calls.

3. The Function Interface in Java

3.1 Purpose

The Function interface, from the java.util.function package, represents a single operation that accepts one input argument and produces one result. It is declared as Function<T, R>, where T is the input type and R is the output type. Its purpose is to let behavior itself be treated as data -- a transformation can be stored in a variable, passed into a method, returned from a method, and reused wherever it is needed. This is the foundation of the functional programming style that Java introduced in version 8.

3.2 Characteristics

- It is a functional interface, meaning it declares exactly one abstract method: R apply(T t). That single-method requirement is what allows it to be implemented with a lambda expression or a method reference instead of a full class.
- It is generic, so the same interface works for any pair of input and output types. In this program it is used as Function<Employee, String>, but it could equally be Function<String, Integer>.
- It provides default methods for composition. The andThen() method chains a second function so the output of the first becomes the input of the second, while compose() performs the same chaining in the opposite order.

- It is meant to be stateless and free of side effects. A Function should compute a result from its input rather than modify anything outside itself, which is what makes it safe to apply across every element of a stream.

Employee Data Processing Using the Function Interface and Streams

3.3 Usage in This Program

In this program the Function is declared as Function<Employee, String> NAME_AND_DEPARTMENT, and its lambda body takes an Employee and returns the employee's name joined to the department. It is then handed directly to the stream's map() operation, which calls apply() on every element and produces a stream of the results. The collect() operation gathers those results into a new List. The original collection is never modified, which is a defining property of stream processing.

The Function declaration and the age-threshold Predicate

```
/**
 * Step 2 requirement: a Function that takes an Employee as input and
 * returns the employee's name and department as a single concatenated
 * String. Function<T, R> means "accepts a T, produces an R".
 */
private static final Function<Employee, String> NAME_AND_DEPARTMENT =
        employee -> employee.getName() + " - " + employee.getDepartment();

private static final Predicate<Employee> ABOVE_AGE_THRESHOLD =
        employee -> employee.getAge() > AGE_THRESHOLD;
```

4. How Streams Complement the Function Interface

A stream is a sequence of elements that supports a pipeline of operations. Intermediate operations such as filter() and map() are lazy -- they describe the work to be done but perform none of it until a terminal operation such as collect(), average(), or findFirst() is invoked. This design lets the whole pipeline run in a single pass over the data instead of building an intermediate collection at every stage.

The program takes advantage of this in several places. The mapToDouble().average() pipeline computes the average salary without ever creating a list of salaries. The summaryStatistics() call produces the count, minimum, maximum, sum, and average together in one traversal instead of running four separate pipelines. Most visibly, findFirst() is short-circuiting: as the program output shows, it stops after examining only three of the eight records once a match is found, rather than processing the entire dataset.

The chained filter -> map -> collect pipeline that applies the age threshold

```
List<String> seniorNameAndDepartment = employees.stream()
        .filter(ABOVE_AGE_THRESHOLD)
        .map(NAME_AND_DEPARTMENT)
        .collect(Collectors.toList());
```

5. Additional Features

Beyond the required operations, I added four features that demonstrate further capabilities of the Function interface and the Stream API:

- Function composition -- andThen() chains the concatenation Function with a String::toUpperCase Function, producing a new Function without modifying either original.
- Grouping with a downstream collector -- Collectors.groupingBy() partitions employees by department while Collectors.averagingDouble() computes each department's average salary in the same single pass.
- Summary statistics -- summaryStatistics() returns the count, minimum, maximum, sum, and average from one traversal, which is more efficient than four separate pipelines.
- A lazy-evaluation demonstration -- a peek() call prints every record the pipeline actually touches, making the short-circuiting behavior of findFirst() visible in the console output rather than only asserted in writing.

6. Program Source Code

Both classes are placed in a single file named EmployeeStreamProcessor.java so the program compiles and runs without any additional project setup.

EmployeeStreamProcessor.java -- the complete program

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.DoubleSummaryStatistics;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.function.Function;
import java.util.function.Predicate;
import java.util.stream.Collectors;

/**
 * EmployeeStreamProcessor.java
 *
 * Demonstrates the java.util.function.Function interface and the Stream API
 * by processing a dataset of employees:
 *   1. Reads the dataset into a collection (List).
 *   2. Uses a Function to concatenate each employee's name and department.
 *   3. Uses streams to build a new collection of those concatenated strings.
 *   4. Calculates the average salary using the stream's built-in functions.
 *   5. Filters the dataset by an age threshold before applying the operations.
 *
 * Additional features are included at the end (function composition, grouping,
 * summary statistics, and a short-circuiting demonstration).
 */
public class EmployeeStreamProcessor {

    /** Age threshold used by the filtering requirement. */
    private static final int AGE_THRESHOLD = 30;

    /**
     * Step 2 requirement: a Function that takes an Employee as input and
     * returns the employee's name and department as a single concatenated
     * String. Function<T, R> means "accepts a T, produces an R" — here it
     * accepts an Employee and produces a String.
     */
    private static final Function<Employee, String> NAME_AND_DEPARTMENT =
            employee -> employee.getName() + " - " + employee.getDepartment();

    /**
     * A reusable Predicate for the age-threshold filter. Keeping it in a named
     * constant makes the stream pipelines below easier to read.
     */
    private static final Predicate<Employee> ABOVE_AGE_THRESHOLD =
            employee -> employee.getAge() > AGE_THRESHOLD;

    public static void main(String[] args) {

        // ---------- 1. Read the dataset and store it in a collection ----------
        List<Employee> employees = loadEmployees();

        printHeader("1. DATASET LOADED INTO A COLLECTION");
        employees.forEach(System.out::println);
        System.out.println("Total records: " + employees.size());
```

```
        // ---------- 2 & 3. Apply the Function via a stream to build a new collection
----------
        // map() applies NAME_AND_DEPARTMENT to every element; collect() gathers
        // the transformed values into a brand-new List, leaving the original
        // collection untouched.
        List<String> nameAndDepartmentList = employees.stream()
                .map(NAME_AND_DEPARTMENT)
                .collect(Collectors.toList());

        printHeader("2. NAME + DEPARTMENT (Function interface applied through a stream)");
        nameAndDepartmentList.forEach(System.out::println);

        // ---------- 4. Average salary using the stream's built-in functions ----------
        // mapToDouble() converts the stream into a DoubleStream so that the
        // built-in average() reduction becomes available. It returns an
        // OptionalDouble because an empty dataset has no average.
        double averageSalary = employees.stream()
                .mapToDouble(Employee::getSalary)
                .average()
                .orElse(0.0);

        printHeader("3. AVERAGE SALARY OF ALL EMPLOYEES");
        System.out.printf("Average salary: $%,.2f%n", averageSalary);

        // ---------- 5. Filter by age threshold, then chain the earlier operations ----------
        // This pipeline chains three stream operations: filter -> map -> collect.
        List<String> seniorNameAndDepartment = employees.stream()
                .filter(ABOVE_AGE_THRESHOLD)
                .map(NAME_AND_DEPARTMENT)
                .collect(Collectors.toList());

        printHeader("4. EMPLOYEES OLDER THAN " + AGE_THRESHOLD + " (filter -> map ->
collect)");
        seniorNameAndDepartment.forEach(System.out::println);

        double averageSalaryAboveThreshold = employees.stream()
                .filter(ABOVE_AGE_THRESHOLD)
                .mapToDouble(Employee::getSalary)
                .average()
                .orElse(0.0);

        System.out.printf("Average salary of employees over %d: $%,.2f%n",
                AGE_THRESHOLD, averageSalaryAboveThreshold);

        // ---------- Additional features ----------
        demonstrateFunctionComposition(employees);
        demonstrateGroupingByDepartment(employees);
        demonstrateSummaryStatistics(employees);
        demonstrateLazyEvaluationAndShortCircuiting(employees);
    }

    /**
     * Builds the employee dataset. In a production system this method would
     * read from a CSV file or a database; the records are hard-coded here so
     * that the program runs anywhere without external dependencies.
     */
    private static List<Employee> loadEmployees() {
        List<Employee> employees = new ArrayList<>();
        employees.add(new Employee("Alice", 28, "Engineering", 72000));
        employees.add(new Employee("Bob", 35, "Marketing", 65000));
        employees.add(new Employee("Charlie", 42, "Engineering", 98000));
        employees.add(new Employee("Diana", 31, "Finance", 81000));
        employees.add(new Employee("Ethan", 26, "Marketing", 54000));
        employees.add(new Employee("Fiona", 45, "Finance", 105000));
        employees.add(new Employee("George", 38, "Engineering", 90000));
        employees.add(new Employee("Hannah", 29, "Human Resources", 61000));
        return employees;
```

```
    }

    /**
     * ADDITIONAL FEATURE 1 — Function composition.
     *
     * andThen() chains two Functions so the output of the first becomes the
     * input of the second. This shows that a Function is a reusable value that
     * can be combined, not just a one-off lambda.
     */
    private static void demonstrateFunctionComposition(List<Employee> employees) {
        Function<String, String> toUpperCase = String::toUpperCase;
        Function<Employee, String> nameDeptUpperCase =
NAME_AND_DEPARTMENT.andThen(toUpperCase);

        printHeader("BONUS 1. FUNCTION COMPOSITION WITH andThen()");
        employees.stream()
                .map(nameDeptUpperCase)
                .forEach(System.out::println);
    }

    /**
     * ADDITIONAL FEATURE 2 — Grouping and downstream collectors.
     *
     * Collectors.groupingBy() partitions the stream into a Map keyed by
     * department, while the downstream collector computes each group's average
     * salary in the same single pass.
     */
    private static void demonstrateGroupingByDepartment(List<Employee> employees) {
        Map<String, Double> averageSalaryByDepartment = employees.stream()
                .collect(Collectors.groupingBy(
                        Employee::getDepartment,
                        Collectors.averagingDouble(Employee::getSalary)));

        printHeader("BONUS 2. AVERAGE SALARY GROUPED BY DEPARTMENT");
        averageSalaryByDepartment.forEach((department, average) ->
                System.out.printf("%-16s : $%,.2f%n", department, average));

        // max() with a Comparator returns an Optional, which safely represents
        // the "no employees" case instead of returning null.
        Optional<Employee> highestPaid = employees.stream()
                .max(Comparator.comparingDouble(Employee::getSalary));

        highestPaid.ifPresent(employee ->
                System.out.println("Highest paid employee: " + employee.getName()));
    }

    /**
     * ADDITIONAL FEATURE 3 — Summary statistics in a single traversal.
     *
     * summaryStatistics() computes count, min, max, sum, and average together,
     * which is more efficient than running four separate stream pipelines.
     */
    private static void demonstrateSummaryStatistics(List<Employee> employees) {
        DoubleSummaryStatistics stats = employees.stream()
                .mapToDouble(Employee::getSalary)
                .summaryStatistics();

        printHeader("BONUS 3. SALARY SUMMARY STATISTICS (single pass)");
        System.out.printf("Count   : %d%n", stats.getCount());
        System.out.printf("Minimum : $%,.2f%n", stats.getMin());
        System.out.printf("Maximum : $%,.2f%n", stats.getMax());
        System.out.printf("Average : $%,.2f%n", stats.getAverage());
    }

    /**
     * ADDITIONAL FEATURE 4 — Lazy evaluation and short-circuiting.
     *
```

```
     * Intermediate operations such as filter() and map() do nothing until a
     * terminal operation runs. findFirst() is short-circuiting, so the pipeline
     * stops as soon as a match is found instead of processing every record.
     * The peek() call prints each element the pipeline actually touches, which
     * makes the saved work visible in the output.
     */
    private static void demonstrateLazyEvaluationAndShortCircuiting(List<Employee> employees)
{
        printHeader("BONUS 4. LAZY EVALUATION AND SHORT-CIRCUITING");

        Optional<String> firstHighEarner = employees.stream()
                .peek(employee -> System.out.println("  examining: " + employee.getName()))
                .filter(employee -> employee.getSalary() > 80000)
                .map(NAME_AND_DEPARTMENT)
                .findFirst();

        System.out.println("First employee earning over $80,000: "
                + firstHighEarner.orElse("none found"));
        System.out.println("Note: the stream stopped early instead of examining all "
                + employees.size() + " records.");
    }

    /**
     * Utility method that prints a consistent section header so the console
     * output stays readable.
     */
    private static void printHeader(String title) {
        System.out.println();
        System.out.println("=".repeat(64));
        System.out.println(title);
        System.out.println("=".repeat(64));
    }
}

/**
 * Employee.java (bundled in the same source file for ease of submission)
 *
 * A simple data class representing one record in the employee dataset.
 * Each Employee has a name, an age, a department, and a salary.
 */
class Employee {

    private final String name;
    private final int age;
    private final String department;
    private final double salary;

    public Employee(String name, int age, String department, double salary) {
        this.name = name;
        this.age = age;
        this.department = department;
        this.salary = salary;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public String getDepartment() {
        return department;
    }

    public double getSalary() {
```

```
        return salary;
    }

    /**
     * Readable representation used when printing an Employee directly.
     */
    @Override
    public String toString() {
        return String.format("%-10s | age %-3d | %-16s | $%,.2f", name, age, department,
salary);
    }
}
```

7. Program Output

The screenshots below show the console output produced by running the program, split across two images for readability.

Console output, part 1 -- dataset, Function applied through a stream, average salary, and the age filter

Console output, part 2 -- function composition, grouping, summary statistics, and short-circuiting

8. Discussion of Results

The output confirms that every required operation produces correct results. All eight employee records are loaded into an ArrayList and printed. The Function is then applied through map() to produce a new collection of eight concatenated strings, while the original list of Employee objects stays unchanged. The average salary across all employees is calculated as $78,250.00 using the stream's built-in average() reduction.

When the age filter is applied, only the five employees older than thirty remain, and the chained filter-map-collect pipeline produces the correct subset of concatenated strings. The average salary of that filtered group rises to $87,800.00, which is consistent with the more senior employees in the dataset holding higher-paying roles.

The additional features behave as expected as well. Function composition applies the concatenation and then converts the result to uppercase without altering the original Function. The grouping collector reports an average salary for each of the four departments in a single pass. The summary statistics section reports five metrics from one traversal. Finally, the short-circuiting demonstration prints each record the pipeline actually examines, making it visible that the stream terminated after three records instead of all eight.

9. How to Run the Program

- Requires a JDK (developed and tested against JDK 21).
- Save the source as EmployeeStreamProcessor.java. The EmployeeStreamProcessor class must appear before the Employee class in the file, since single-file source mode runs whichever class is declared first.
- From a terminal in the same folder, run "java EmployeeStreamProcessor.java" to compile and run in one step, or "javac EmployeeStreamProcessor.java" followed by "java EmployeeStreamProcessor".
- The program prints all output sections in order and exits; no user input is required.

10. Conclusion

This assignment demonstrates that the Function interface and the Stream API work together to express data transformations declaratively rather than through manual iteration. The Function interface captures a single input-to-output transformation as a reusable value, while streams

provide the pipeline through which that transformation is applied across a collection. Together they produce code that states what should happen to the data rather than how to loop over it, and the stream's lazy evaluation and short-circuiting behavior mean that this clarity does not come at the cost of performance.

References

Eck, D. J. (2022). *Introduction to programming using Java* (Version 9, JavaFX edition). https://math.hws.edu/javanotes/

Oracle. (n.d.). *Interface Function<T,R>*. Java Platform SE API Specification. https://docs.oracle.com/javase/8/docs/api/java/util/function/Function.html